Name: Harshita Singh
Class: D20C
Roll no. : 33
Subject: BlockChain

Experiment No: 8

Aim

Configuring a cryptocurrency wallet using MetaMask and integrating it with a decentralized application (DApp).

Theory

The Problem in Retail and E-Commerce

The retail and e-commerce sector involves multiple participants, including manufacturers, distributors, retailers, and customers. Customers often rely on product descriptions, seller claims, and warranty information to determine whether a product is genuine. Traditional systems maintain product information in centralized databases, making it difficult for customers to independently verify product records across different sellers and supply-chain participants. This can lead to counterfeit products, unreliable product information, and difficulties in tracing product origins.
Blockchain technology provides a shared, tamper-resistant ledger for maintaining product records. A blockchain-based product verification system can store product identifiers, manufacturer addresses, registration timestamps, and references to product metadata. These records can be retrieved by users to check whether a product has been registered and to inspect its associated information.
TrustCart is a blockchain-based product registration and verification prototype designed to demonstrate how blockchain can improve transparency and traceability in retail and e-commerce.

MetaMask and Decentralized Applications (DApps)

MetaMask is a cryptocurrency wallet that enables users to manage blockchain accounts, connect to Ethereum-compatible networks, and authorize blockchain transactions. It manages the user's private keys and requests approval before signing transactions.
A decentralized application (DApp) is an application that provides a user interface for interacting with smart contracts deployed on a blockchain. In this experiment, TrustCart is developed using HTML, CSS, and JavaScript. MetaMask provides wallet connectivity and transaction authorization, allowing the frontend to interact with the deployed smart contract.

Integration Using Ethers.js

Ethers.js is a JavaScript library that enables web applications to communicate with Ethereum-compatible blockchains and interact with deployed smart contracts. The library uses the smart contract's address and Application Binary Interface (ABI) to call contract functions.
The TrustCart frontend connects to MetaMask through the browser's Ethereum provider, `window.ethereum`. Ethers.js uses the provider to access blockchain information and a wallet signer to submit transactions. MetaMask requests user approval before signing transactions that modify blockchain state. These transactions require gas fees paid using the network's cryptocurrency.
Read-only operations, such as retrieving a registered product's details, can be performed without submitting a transaction or paying transaction gas fees.

Design of the ProductPassport Smart Contract

The `ProductPassport` smart contract is developed using Solidity to maintain product registration records and support product verification on the Ethereum blockchain.

- **struct Product:** Stores the product ID, product name, metadata identifier, manufacturer's wallet address, and registration timestamp.
- **mapping(uint256 => Product):** Stores product records using the product ID as the key.
- **mapping(uint256 => bool) productExists:** Tracks whether a product ID has already been registered.
- **registerProduct():** Registers a new product on the blockchain and records the caller's wallet address as the manufacturer.
- **verifyProduct():** Retrieves the details of a registered product, including its name, metadata identifier, manufacturer address, and registration timestamp.
- **productExists():** Checks whether a product record exists for a specified ID.
- **ProductRegistered event:** Emits an event when a product is registered, making the registration activity available for transaction and event inspection.
- **require() checks:** Validate that the product ID is non-zero, the product ID has not already been registered, and the product name and metadata identifier are not empty.

These rules prevent duplicate product registrations and invalid product entries. The contract maintains the product information on the blockchain, allowing users to retrieve the registered details.

The prototype verifies the existence of a blockchain record; it does not independently establish the physical authenticity of a product. This depends on the reliability of the information entered during registration.

Ethereum Sepolia Testnet

Ethereum Sepolia is a test network used to develop and test Ethereum applications without using real-value Ether. In this experiment, the `ProductPassport` contract is deployed to Sepolia through Remix IDE using MetaMask.

Sepolia test ETH is obtained from a faucet and is used to pay transaction gas fees. The deployed contract address is configured in the TrustCart frontend so that the application can interact with the correct contract on Sepolia. Sepolia Etherscan is used to inspect deployment and product-registration transactions.

Working of the TrustCart DApp

The TrustCart DApp connects the user's MetaMask wallet to the Ethereum Sepolia testnet. Ethers.js enables the frontend to call functions of the deployed `ProductPassport` smart contract.

The user first connects the wallet to the application. To register a product, the user enters a product ID, product name, and metadata identifier. The DApp submits the registration transaction, which the user approves through MetaMask. After successful confirmation, the product record is stored on the blockchain.

To verify a product, the user enters its product ID. The application calls `verifyProduct()` and displays the registered product details. The transaction hash can be inspected using Sepolia Etherscan to verify the blockchain transaction.

This integration demonstrates how MetaMask, Ethers.js, a web-based interface, and a Solidity smart contract work together to provide a basic blockchain-based product registration and verification system.

Smart Contract Code

The `ProductPassport.sol` smart contract is implemented in Remix IDE. It defines the product structure, registration mappings, validation rules, registration event, and functions for registering and verifying products.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract ProductPassport {
    struct Product {
        uint256 productId;
        string productName;
        string metadataHash;
        address manufacturer;
        uint256 registeredAt;
    }

    mapping(uint256 => Product) private
products;
    mapping(uint256 => bool) public
productExists;

    event ProductRegistered(
        uint256 indexed productId,
        string productName,
        string metadataHash,
        address indexed manufacturer,
        uint256 registeredAt
    );

    function registerProduct(
        uint256 _productId,
        string memory _productName,
        string memory _metadataHash
    ) public {
        require(_productId != 0,
"Invalid product ID");

require(!productExists[_productId],
                "Product already
registered");

require(bytes(_productName).length > 0,
                "Product name
required");

require(bytes(_metadataHash).length >
0,
                "Metadata hash
required");

        products[_productId] =
Product({
            productId: _productId,
            productName: _productName,
            metadataHash:
_metadataHash,
            manufacturer: msg.sender,
            registeredAt:
block.timestamp
        });

        productExists[_productId] =
true;

        emit ProductRegistered(
            _productId,
            _productName,
            _metadataHash,
            msg.sender,
            block.timestamp
        );
    }

    function verifyProduct(uint256
_productId)
        public
        view
        returns (
            bool exists,
            string memory productName,
            string memory metadataHash,
            address manufacturer,
            uint256 registeredAt
        )
    {

require(productExists[_productId],
                "Product not found");

        Product memory p =
products[_productId];

        return (
            true,
            p.productName,
            p.metadataHash,
            p.manufacturer,
            p.registeredAt
        );
    }
}
```

DApp Code

The TrustCart DApp is implemented using HTML, CSS, JavaScript, and Ethers.js. The frontend includes wallet connection, product registration, product verification, transaction status display, and a link to inspect transactions on Sepolia Etherscan.

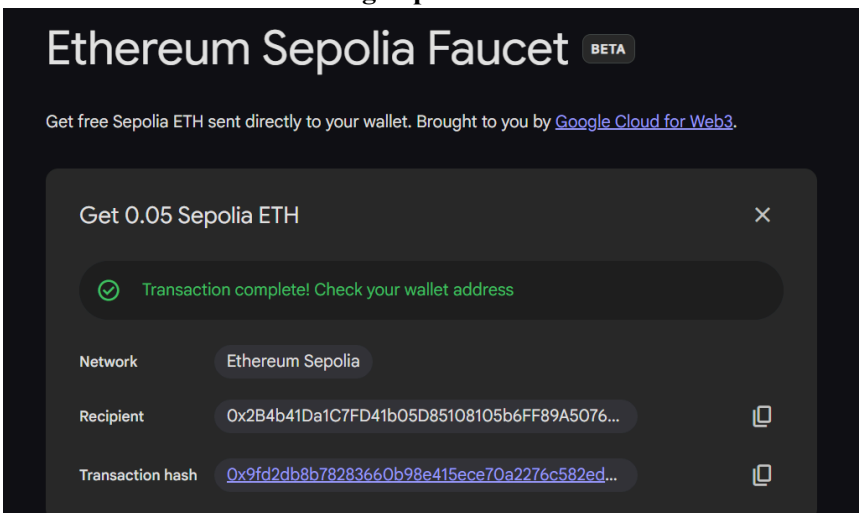
The frontend source code is maintained in the `index.html` file of the TrustCart experiment folder. The deployed `ProductPassport` contract address is configured in the frontend to connect the application to the Sepolia smart contract.

Procedure

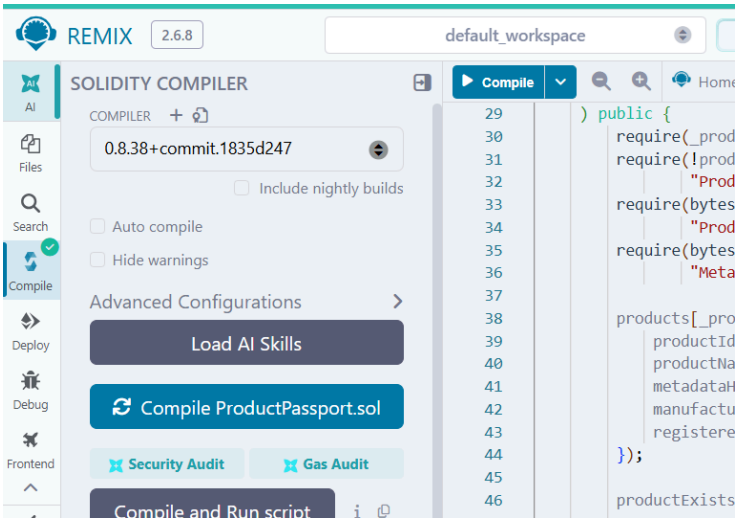
1. Installed and configured the MetaMask browser extension.
2. Enabled the Ethereum Sepolia test network in MetaMask.
3. Obtained Sepolia test ETH from a faucet for deployment and transaction gas fees.
4. Created `ProductPassport.sol` in Remix IDE and compiled it using a compatible Solidity compiler.
5. Selected the Sepolia MetaMask browser-wallet environment in Remix and connected the wallet.
6. Deployed the smart contract to Sepolia and approved the deployment transaction through MetaMask.
7. Copied the deployed contract address and configured it in the TrustCart frontend.
8. Created the `index.html` frontend using HTML, CSS, JavaScript, and Ethers.js.
9. Launched the frontend locally using VS Code Live Server.
10. Connected the TrustCart DApp to MetaMask and verified that the wallet was connected to Sepolia.
11. Registered a product by entering its product ID, name, and metadata identifier.
12. Approved the registration transaction through MetaMask and waited for confirmation.
13. Verified the registered product through the frontend using its product ID.
14. Opened the transaction hash on Sepolia Etherscan to inspect the transaction details.

Output

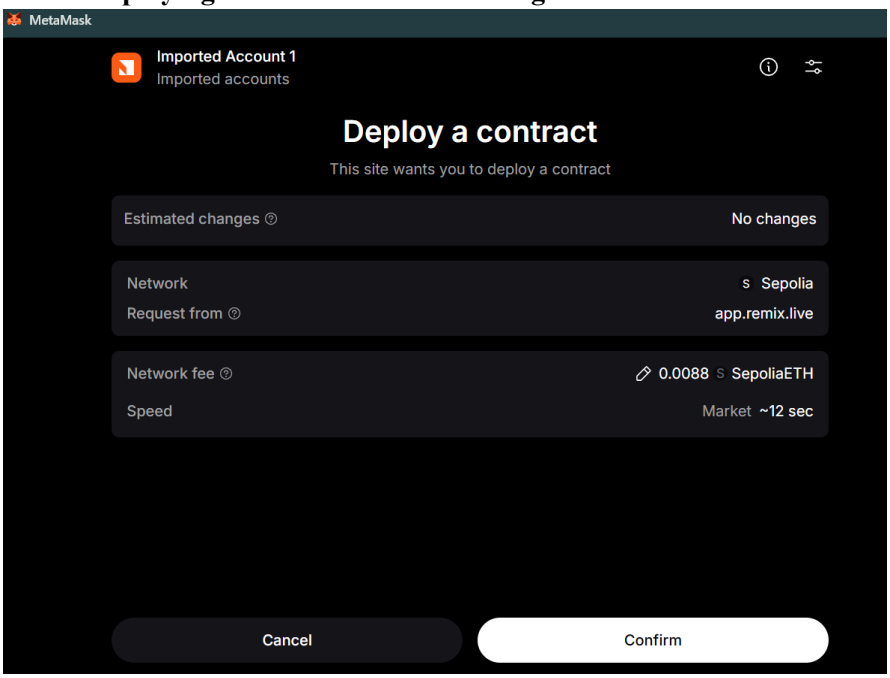
1. Obtaining Sepolia Testnet ETH



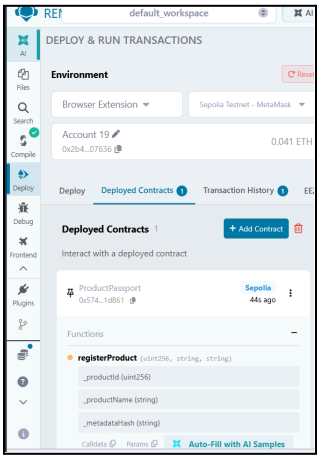
2. Creating and Compiling the Smart Contract

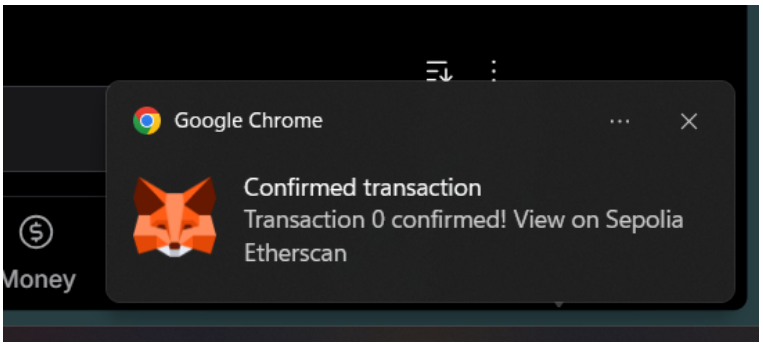


3. Deploying the Smart Contract Using Remix and MetaMask

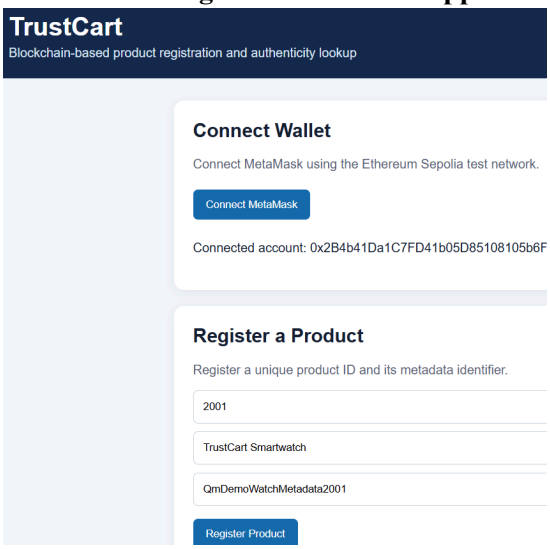


4. Deployed Smart Contract

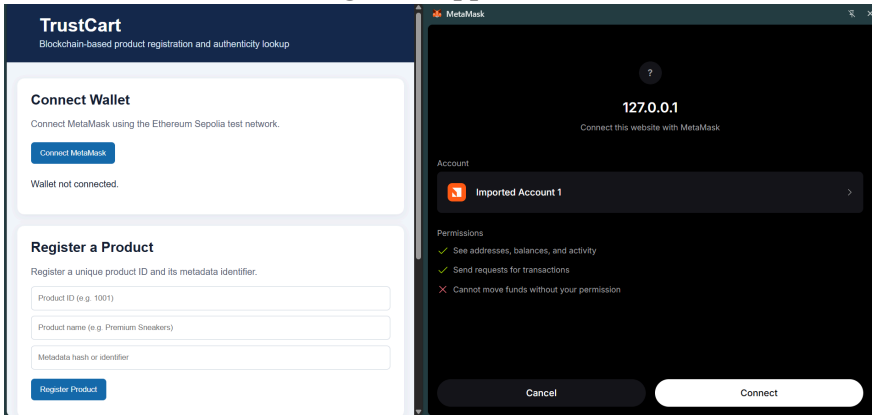




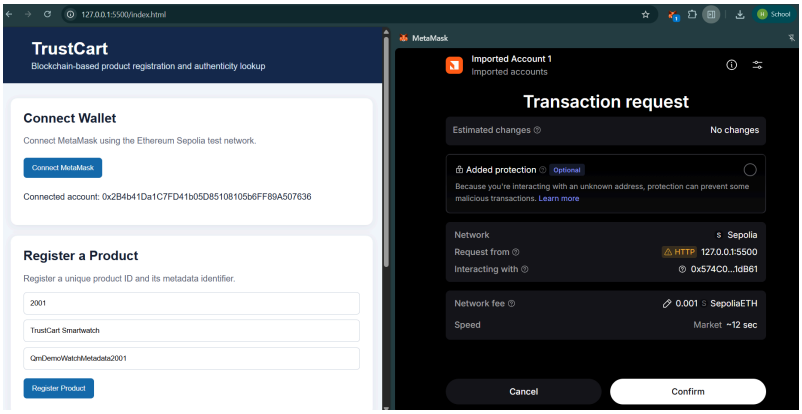
5. Creating the TrustCart DApp



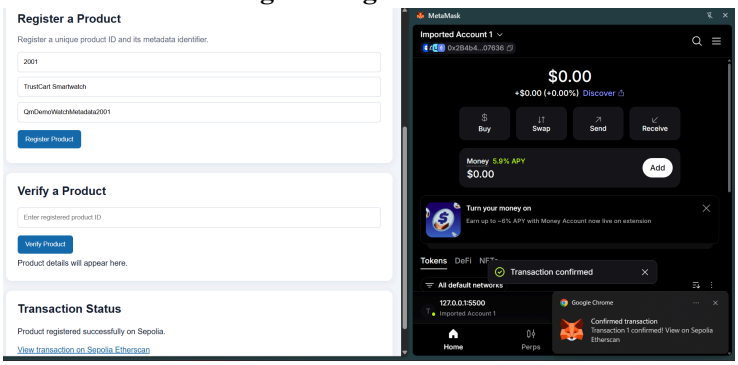
6. Connecting the DApp with MetaMask



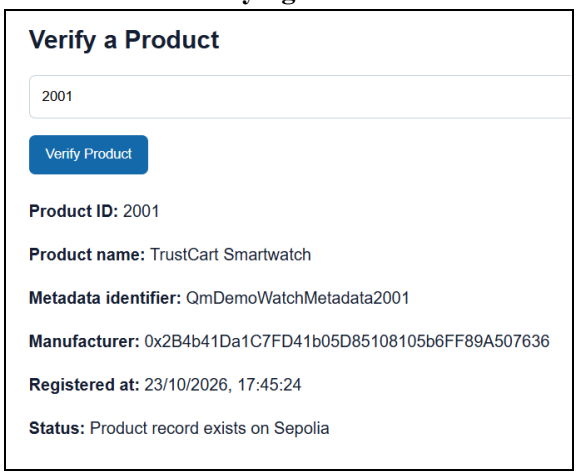
7. Registering a Product



8. Confirming the Registration Transaction



9. Verifying a Product



Conclusion

MetaMask was successfully integrated with the TrustCart DApp developed using HTML, CSS, JavaScript, and Ethers.js. The `ProductPassport` smart contract was deployed on the Ethereum Sepolia testnet using Remix IDE. The application supports product registration and retrieval of registered product details through a web-based interface. MetaMask enables wallet connection and transaction authorization, while Ethers.js facilitates communication between the frontend and the smart contract. Sepolia Etherscan provides a means of inspecting the deployment and registration transactions. The experiment demonstrates how blockchain smart contracts can support transparent product-record management and basic product verification in the retail and e-commerce sector.